

SUMMER 2026



LONGHORN HAPKIT

Lab guide #2 - morning

ROSS NEUMAN

Assistant Professor of Practice, The University of Texas at Austin

After springs, bumps, and walls

- So far all of our commanded forces have been related to the **position** of the end-effector (handle)
- Sometimes we want to instead have them relate to the **velocity** of the handle

$$\textit{velocity} = \frac{\textit{position}}{\textit{time}}$$

How do we get the time?

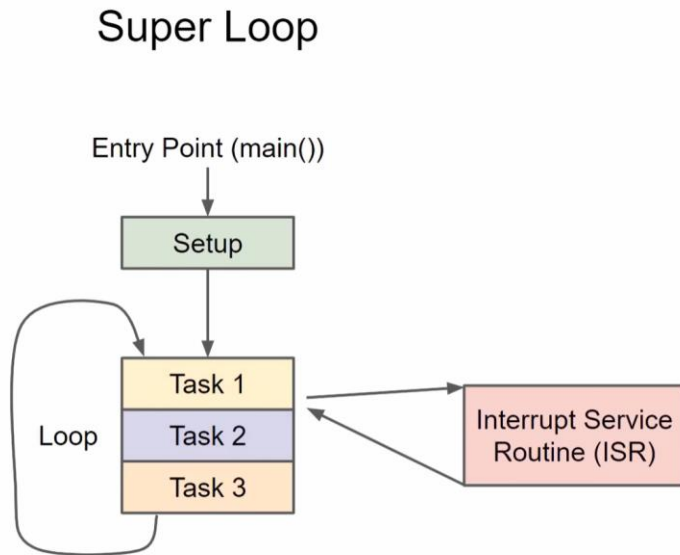
- Currently our superloop simply runs as fast as it can
- This means we don't have a good measure of how much time elapses between runs
- Solution: we make another loop that *does* run at a given time

Interrupt Service Routine (ISR)

- ISR uses a *timer* onboard the microcontroller and counts for a certain number of clock cycles
- Once it hits the number, it triggers the ISR to run **immediately**

How are we handling haptic loop now?

- Rather than running each loop task sequentially, ISR initiates at precise time intervals
 - Processor finishes current instruction
 - Jumps to ISR
- Only preempted by higher-priority interrupts like...?
- Encoder!
 - This is fine because encoder is super fast



Haptic loop handling – ISR setup

```
cli(); //disable interrupts

TCCR2A = 0;
TCCR2B = 0;

// waveform generation mode 2 -- clear timer on compare match (CTC) mode
TCCR2A |= (1 << WGM21);

// Prescaler = 64
TCCR2B |= (1 << CS22);

OCR2A = 249;
// freq = freq_cpu / (prescaler * (1 + OCR2A))
// freq = 16,000,000 / (64 * (1 + 249)) = 1000 Hz or 1 kHz
// no multiplication by factor of 2 in denominator as Timer2 is single-slope (Timer1 is dual-slope)

// enable interrupt
TIMSK2 |= (1 << OCIE2A);

sei(); // enable interrupts
```

Haptic Loop Handling - ISR

- ISR is a separate loop from the “superloop”
- Use this for computation of quantities necessary to haptic rendering
- **Do not use this for things like printing to serial monitor**
 - Serial prints are very slow
 - This is what the regular loop() function can be used for now

```
//Haptic loop running at 1kHz using an interrupt service routine (ISR) for reliable calling
ISR(TIMER2_COMPA_vect) {

    // read encoder
    pos = encoder.read();

    tPulley = (pos / encoderResolution) * 2 * PI; //angular position of motor pulley in radians

    tS = (tPulley * rp) / rs; //angular position of sector pulley in radians

    //determine handle position in cm
    xh = (rh * rp * tPulley) / rs;

    // handle velocity (cm/s) using a 1st-order low pass filter with constant r defined in setup()
    vh = r * vh_prev + (1 - r) * (xh - xh_prev) / dt;

    // update "previous" terms for next loop
    xh_prev = xh;
    vh_prev = vh;

    // command motor based on duty cycle (-255 to 255)
    setMotor(int(motorCommand));
}
```

ISR – The catch

- What happens if an interrupt is triggered before the ISR is finished executing?
- It triggers the ISR inside of the ISR
- System can very quickly go **unstable**
- This is why one should keep the code within the ISR lean and straightforward
 - In our case it needs to execute in 1ms or 16,000 clock cycles
 - Serial calls or lots of FLOPs can make this a reality

How the new code flows:

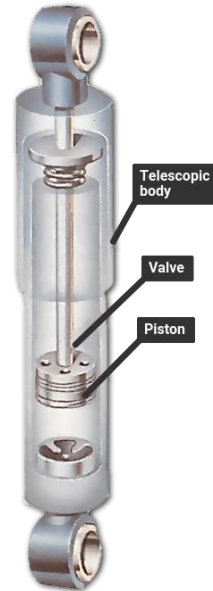
- **No need to touch any of the ISR stuff**
- Changes you make should occur in elsewhere:
 - Haptic modes in loop()
 - Serial communications in loop()
 - Variable declarations above

Springs and Dampers

- Spring force is a **restoring** force that tries to pull an object back to equilibrium
 - Follows Hooke's law: $F_s = -kx$
 - k is spring constant
 - x is how far string has been stretched
 - Only relies on object's position
- Damping force is a **dissipative** force that tries to stop motion by pulling energy out of it
 - Follow the equation $F_d = -b\dot{x}$
 - Where b is the damping coefficient (viscosity), and
 - $\dot{x}$ is the velocity of the object (time derivative of position)

Springs and Dampers

- Often times real-world systems/interactions exhibit both types of force
- Consider an automotive suspension



Underdamped, Overdamped, Critically Damped

- The **system dynamics** that result are relevant to all sorts of systems
 - Mechanical
 - Electrical (RC circuits)
 - Financial
 - Population dynamics



Dudley Thompson

Creating damping in haptic devices

- Creating a damper is relatively simple, as you just need a damping term and the velocity:

$$F_d = -b\dot{x}$$

- However, determining the velocity requires care

Velocity Estimation

- Need to take a time derivative of positions
 - We have positions (encoder)
 - We have idea of time (haptic loop at 1 kHz)
- However, differentiation of encoder positions creates/amplifies **noise**
- Filtering is required to get smooth, usable values of velocity
 - Filtering introduces delays, reducing transparency
 - Must find a good balance

Method 1: Raw Differentiation

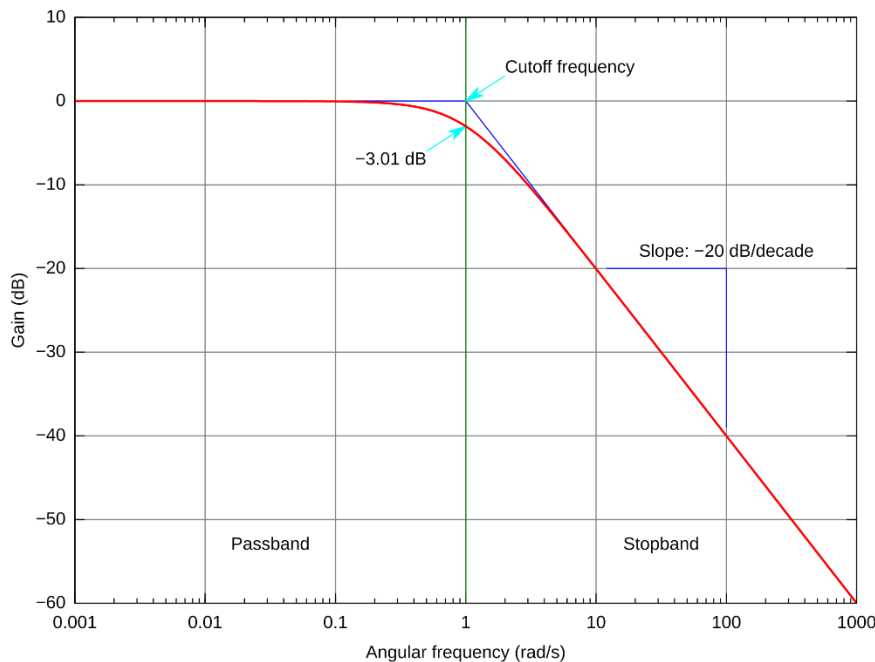
- $v_k = \frac{dx}{dt} = \dot{x} = \frac{(x_k - x_{k-1})}{dt}$
 - x_k is current handle position
 - x_{k-1} is position of handle at the last time step
 - dt is the time step
 - Our haptic loop is 1 kHz, meaning dt is .001s or 1ms
- This approach is very noisy and pretty much unusable for force feedback devices
- Let's code it out

Method 2: Windowed Velocity

- $v_k = \frac{dx}{dt} = \dot{x} = \frac{(x_k - x_{k-N})}{N * dt}$
- Creates a “moving window” that averages out over N samples
- Pros:
 - In this approach you have much lower noise
 - Predictable and reliable
- Cons:
 - Introduces delay of $\sim$ window size/2
 - Larger windows make “softer” haptic response
 - Windows above 10ms are too mushy

Method 3: Low-Pass Differentiator

- $$v_k = \frac{rv_{k-1} + (1-r)(x_k - x_{k-1})}{dt}$$
 - Where $r = e^{-2\pi f_c dt}$
 - Where f_c is the desired **cutoff frequency**
- Pros:
 - Good balance between noise and delay
 - Can tune easily by adjusting f_c
- Cons:
 - Sometimes still harsh or unstable
- Try to implement this in your code!
 - Let's talk about the pseudocode
 - Can just use 0.95 for “r” for now
 - Try f_c values anywhere from 1-60hz or so and see what works best!



Example low pass filter

Velocity filtering

- This is why we need to filter our velocity
- The code provided implements a 1st-order, discrete time low-pass filter:

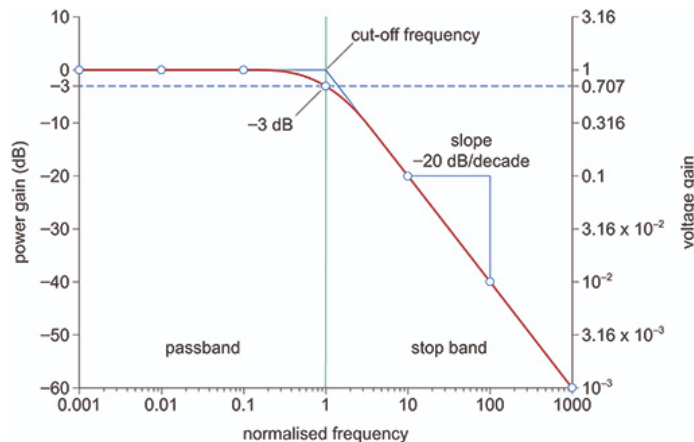
$$v_h[k] = r v_h[k - 1] + (1 - r) \frac{x_h[k] - x_h[k - 1]}{\Delta t}$$

Where

$$r = e^{-2\pi f_c \Delta t}$$

Velocity filtering

- When tuning your velocity filter, try changing the value of **fc** in the loop— this is your **cutoff frequency**
- The higher your cutoff frequency, the more high frequencies will be “passed” through



Damper

- Create a damper and mess with **fc** as well as **b** to find what kinds of forces you can render now
- Damper is included in the newly uploaded code

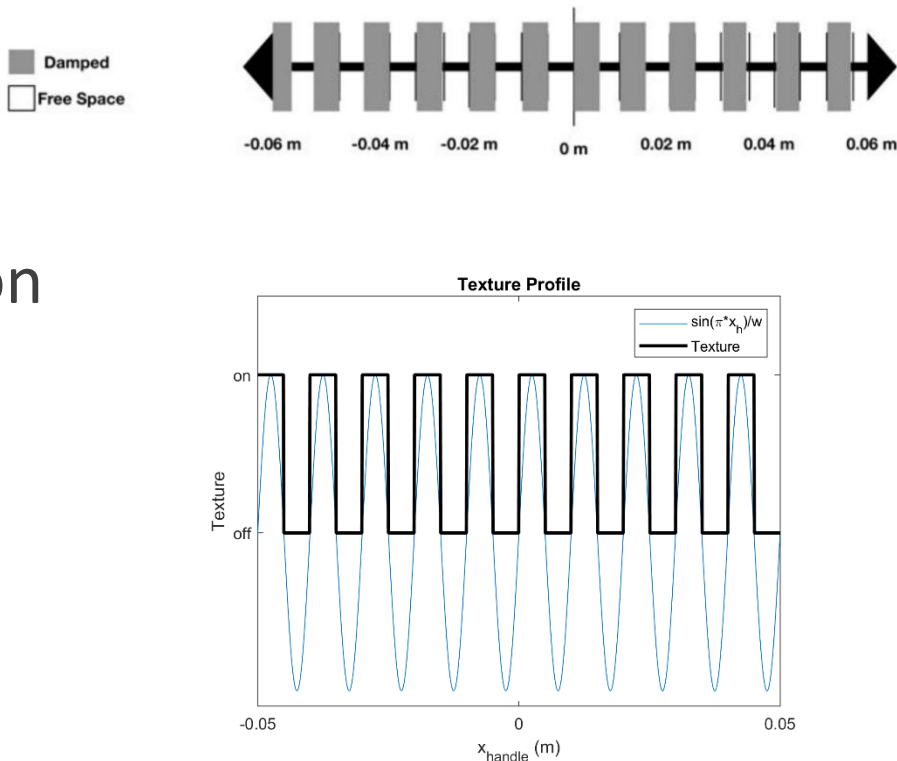
```
// Virtual damper
case '3':
{
    const float b = 8;           //damping coefficient
    motorCommand = -b * vh;      //multiplied by velocity
    break;
}
```

Spring+Damper

- Add together the effects of a spring and a damper to make a new mode
- See if you can make it go unstable
 - If you do make things go unstable, careful not to burn your motor up! Unplug before it's too late!

Texture

- Alternate sections of damping and no damping to create a textured illusion
 - Can do this with conditionals
 - Can do it with modulus
 - Can do using a sinusoid—changing period lets you change “coarseness” of texture



Add damping to a wall

- Sometimes this helps reject instabilities with high stiffness walls

Continue Developing Virtual Environments

- If you are through with everything, get creative and mix and match these ideas, or come up with your own
- It's possible to even do simple dynamic simulations

If you are tired of making virtual environments

- Try out Processing
- <https://processing.org/tutorials/>
- See if you can use it to create real-time graphics related to the Hapkit